

Výstupy ŠVP

INF-08c V programu používám podprogramy s parametry a návratovými hodnotami. (*Informatika 4, 2/2 · z INF-08*)**INF-07a** Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešení problém ten nejvhodnější. (*Informatika 4, 2/2 · z INF-07*)**Učivo:** programovací koncepty – podprogramy s parametry a s návratovými hodnotami

Připomenutí

$$n! = n \cdot (n-1)!, \quad 0! = 1$$

$$\binom{n}{k} = \frac{n!}{k! (n-k)!}$$

$$\binom{n}{k} = \frac{n(n-1) \dots (n-k+1)}{k!}$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Podprogramy (funkce) s parametry a návratovou hodnotou: faktoriál, kombinační čísla, opakované použití funkcí a porovnání cyklu s rekurzí podle počtu kroků. Podrobnosti najdeš v listu *Must know*.

A. Rozcvička

1. Co program vypíše? ★☆☆

```
def dvojnásobek(x):
    return 2 * x

a = dvojnásobek(5)
print(a, dvojnásobek(a), dvojnásobek("ha"))
```

2. Co program vypíše? Proč je ve druhém řádku None? Oprav funkci. ★☆☆

```
def obvod(a, b):
    print(2 * (a + b))

o = obvod(3, 4)
print("Obvod je", o)
```

3. Které jsou parametry a které argumenty? Kolikrát se funkce zavolá a co program vypíše? ★☆☆

```
def obsah(a, b):
    return a * b

s1 = obsah(3, 4)
s2 = obsah(s1, 2)
print(s1 + s2, obsah(2, obsah(1, 5)))
```

4. Co program vypíše? Co by se změnilo, kdyby řádek return None byl odsazen uvnitř cyklu? ★☆☆

```
def prvni_sude(seznam):
    for x in seznam:
        if x % 2 == 0:
            return x
    return None

print(prvni_sude([3, 7, 8, 5, 10]), prvni_sude([1, 3]))
```

5. Co program vypíše? Proč třetí řádek skončí chybou a proč se hodnota n mimo funkci nezměnila? ★☆☆

```
def zvys(n):
    n = n + 1
    vysledek = n * 10
    return vysledek

n = 5
print(zvys(n), n)
print(vysledek)
```

6. Spočítej bez počítáče:
- $5!$
- ,
- $0!$
- ,
- $\binom{6}{2}$
- ,
- $\binom{10}{0}$
- ,
- $\binom{10}{10}$
- ,
- $\binom{7}{3}$
- a
- $\binom{7}{4}$
- . ★☆☆

7. Rozepiš, jak se postupně volá rekurzivní funkce. Co vypíše a kolikrát se funkce f zavolá? ★★☆☆

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)

print(f(4))
```

8. Doplň tělo cyklu ve funkci faktorial a otestuj ji pro $n = 0, 1, 6$. ★☆☆☆

```
def faktorial(n):
    vysledek = 1
    for i in range(2, n + 1):
        ...
    return vysledek
```

B. Slovní úlohy

9. Napiš funkci kombinace(n, k) s využitím funkce faktorial a spočítej: kolika způsoby lze vybrat 3 žáky z 10; kolik různých dvojic lze vytvořit ve třídě 30 žáků; kolik je možných tipů ve hře 6 čísel ze 49. ★☆☆☆
10. Přepiš výpočet průměru a maxima z H49 do funkcí prumer(s) a maximum(s). Použij je pro známky ze tří předmětů: matematika [1, 2, 2, 3], fyzika [2, 2, 1], informatika [1, 1, 2, 1]. Co má vrátit prumer([])? ★★☆☆
11. Napiš funkci je_prvocislo(n), která vrací True, nebo False (stačí zkoušet dělitele d , dokud $d \cdot d \leq n$). Pomocí ní vypiš všechna prvočísla do 50 a jejich počet. ★★☆☆
12. Přepiš řešič z H44 na funkci koreny(a, b, c) pro $a \neq 0$, která vrací seznam kořenů (prázdný, jednoprvkový, nebo dvouprvkový). Napiš k ní tři testy assert a zjisti, kolik kořenů má $2x^2 - 4x - 6 = 0$. ★★☆☆
13. Pomocí funkce kombinace a dvou cyklů vypiš prvních 6 řádků Pascalova trojúhelníku ($n = 0$ až 5). ★★☆☆
14. Uprav funkci faktorial tak, aby kromě výsledku vracela i počet provedených násobení (return vysledek, nasobeni). Kolik násobení potřebuje pro $n = 10, 100, 1\,000$? Jak roste počet kroků s n ? ★☆☆☆
15. Napiš funkci kombinace2(n, k) podle součinného vzorce: $c = 1$ a pro i od 1 do k provedeš $c = c * (n - k + i) // i$. Ověř, že dává stejné výsledky jako kombinace(n, k). Porovnej počet násobení pro $\binom{1000}{3}$. ★★☆☆
16. Rozklad programu na podprogramy: Program z H49 (úloha 10) načítá časy běhu a vypisuje statistiku. Navrhni jeho rozdělení na funkce (název, parametry, návratová hodnota) tak, aby šly použít i pro jiná data, a napiš hlavní část programu, která je volá. ★★☆☆

C. Problémy a propojení

17. Cyklus $\times$ rekurze: Rekurzivní funkce počítá kombinační čísla podle Pascalova trojúhelníku a počítá svá volání. Co program vypíše? Proč počet volání tak rychle roste a jak by vyšel pro $\binom{26}{13}$? Který algoritmus pro kombinace bys zvolil? ★★☆☆

```
volani = 0
def komb_r(n, k):
    global volani
    volani += 1
    if k == 0 or k == n:
        return 1
    return komb_r(n - 1, k - 1) + komb_r(n - 1, k)

for n, k in ((10, 5), (20, 10)):
    volani = 0
    print(komb_r(n, k), volani)
```

18. Rekurse a paměť: Rekurzivní faktoriál z úlohy 7 spočítá $f(900)$, ale $f(5000)$ skončí chybou. Jakou a proč? Proč verze s cyklem z úlohy 8 funguje? Porovnej oba algoritmy podle počtu kroků a potřebné paměti. ★★☆☆
19. Matematika $\times$ informatika (H51): Koeficienty rozvoje $(a + b)^n$ jsou čísla $\binom{n}{k}$. Pomocí funkce kombinace vypiš koeficienty rozvoje $(x + 1)^5$ a automaticky ověř (assert), že součet čísel v každém řádku Pascalova trojúhelníku je 2^n pro n od 0 do 20. Proč to platí? ★★☆☆

20. Odhad: Playlist má 20 skladeb. Kolik existuje různých pořadí? Kdyby počítač vyzkoušel miliardu pořadí za sekundu, jak dlouho by je procházel všechna? Odhadni, pak spočítej funkci faktorial. ★★☆☆

Hotové úlohy k procvičení na internetu

- **Nauč se Python (PyLadies) – Vlastní funkce** – def, parametry, return, opakované volání; úkol s řešením
<https://naucese.python.cz/course/pyladies/beginners/def/>
- **Nauč se Python (PyLadies) – Lokální proměnné** – proč proměnné z funkce nejsou vidět venku
<https://naucese.python.cz/course/pyladies/beginners/local-variables/>
- **iMyšlení – Základy programování v jazyce Python pro SŠ** – učebnice schválená MŠMT, lekce Podprogramy a Podprogram s parametrem; pracovní listy ke stažení
<https://archiv-imysleni.npi.cz/ucebnice/zaklady-programovani-v-jazyce-python-pro-stredni-skoly.html>

★ základní · ★★ střední · ★★★ náročnější. Vyber si, které úlohy spočítáš; řešení ukládej do svého portfolia.

Výsledky ke kontrole

- 10 20 haha
- 14 a Obvod je None – funkce výsledek jen vypíše, nevrátí; oprava: `return 2 * (a + b)` → Obvod je 14
- parametry a, b; argumenty 3, 4, s1, 2, 1, 5, 2 a výsledek vnitřního volání; 4 volání; vypíše 36 10
- 8 None; s `return None` v cyklu by funkce skončila už po prvním prvku (3) a vrátila None i pro první seznam
- 60 5, pak `NameError: name 'vysledek' is not defined` – výsledek i parametr n jsou lokální proměnné funkce; vnější n zůstalo 5
- 120; 1; 15; 1; 1; 35 a 35 (platí $\binom{n}{k} = \binom{n}{n-k}$)
- $f(4) = 4 \cdot f(3) = 4 \cdot 3 \cdot f(2) = \dots = 4 \cdot 3 \cdot 2 \cdot 1 \cdot f(0) = 24$; 5 volání ($f(4)$ až $f(0)$)
- `vysledek = vysledek * i`; `faktorial(0) = 1`, `faktorial(1) = 1` (cyklus neproběhne ani jednou), `faktorial(6) = 720`
- 120; 435; 13 983 816
- průměry 2,0; 1,67; 1,25; maxima 3; 2; 2; pro prázdný seznam vrátit None (nebo upozornění) místo dělení nulou – každá funkce je použita třikrát s jinými daty
- 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47 – celkem 15; pozor na $n = 0$ a 1 (nejsou prvočísla)
- `assert koreny(1, -5, 6) == [3.0, 2.0]`; `assert koreny(1, 2, 1) == [-1.0]`; `assert koreny(1, 0, 1) == []`; `len(koreny(2, -4, -6)) = 2` → `[3.0, -1.0]`
- 1 / 1 1 / 1 2 1 / 1 3 3 1 / 1 4 6 4 1 / 1 5 10 10 5 1
- 9; 99; 999 – počet kroků je $n - 1$, roste lineárně ($10 \times$ větší vstup → $10 \times$ víc násobení)
- `kombinace2(10, 3) = 120`, `(49, 6) = 13 983 816`, `(1000, 3) = 166 167 000`; přes faktoriály $999 + 2 + 996 = 1 997$ násobení (s obrovskými čísly), součinně jen 3 násobení a 3 dělení; dělení vychází beze zbytku, protože po i -tém kroku je $c = \binom{n-k+i}{i}$
- např. `nacti_cisla()` → seznam (while do nuly); `prumer(s)` → float nebo None; `minimum(s)` → číslo; `vypis_statistiku(s)` → nic (jen výpis); hlavní část: `casy = nacti_cisla()`; `vypis_statistiku(casy)` – funkce `prumer` a `minimum` jsou použitelné i pro známky nebo teploty
- 252 503 a 184756 369511 – počet volání je $2 \binom{n}{k} - 1$, protože rekurze sčítá samé jedničky a stejné hodnoty počítá stále znovu; pro $\binom{26}{13} = 10 400 600$ asi 20,8 milionu volání (sekundy); nejvhodnější je součinný vzorec (úloha 15) s k kroky
- `RecursionError: maximum recursion depth exceeded` – každé rozpracované volání zabírá místo v paměti (zásobník), Python povolí asi 1 000 úrovní; cyklus potřebuje jen pár proměnných; počet násobení je u obou stejný ($n - 1$, resp. n), liší se paměť
- 1 5 10 10 5 1; test projde pro všechna n ; dosazení $a = b = 1$ do binomické věty:
$$2^n = (1 + 1)^n = \sum_{k=0}^n \binom{n}{k}$$
- $20! \doteq 2,43 \cdot 10^{18}$ pořadí; $2,43 \cdot 10^9 \doteq 77$ let – kombinatorická exploze (H50): úlohy „vyzkoušej všechny možnosti“ jsou pro velká n nepoužitelné

Řešení krok za krokem

Úloha 17

- Globální proměnná volání se zvýší při každém zavolání `komb_r`; před každým výpočtem ji program vynuluje.
- Funkce počítá $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ a končí na okrajích trojúhelníku ($k = 0$ nebo $k = n$), kde vrací 1.
- Výsledek je tedy součet samých jedniček: strom volání má $\binom{n}{k}$ listů a o jeden méně vnitřních uzlů → $2 \binom{n}{k} - 1$ volání.
- Výpis: 252 503 ($2 \cdot 252 - 1 = 503$) a 184756 369511 ($2 \cdot 184 756 - 1 = 369 511$).
- Stejně hodnoty se počítají pořád znovu, proto počet volání roste stejně rychle jako samo kombinační číslo.
- $\binom{26}{13} = 10 400 600$ → asi 20,8 milionu volání (sekundy); součinný vzorec z úlohy 15 potřebuje jen $k = 13$ kroků, a proto je nejvhodnější.